Open in app ↗ Sign up Sign in

Medium Search

OLX Engineering · Follow publication

Optimizing FastAPI at Scale: Lessons from OLX’s Recommendation Platform

10 min read · Jul 8, 2025

Jordi Esteve Sorribas Follow

Listen Share

Photo by Rosy Ko

> *In distributed systems, there is a motto that says 'you are as slow as your slowest tasks'. In Python, thanks to the notorious Global Interpreter Lock (GIL), this issue is amplified: 'your slowest task will make every other task slower'. In this article, I'll walk you through the optimizations we made to scale a FastAPI service that now handles tens of thousands of requests per second, achieving a* ***p99 latency under 10ms****.*

Introduction

OLX is a global online marketplace that enables users to buy and sell goods and services, primarily through classified ads. We have a clear vision: to create leading marketplace ecosystems enabled by tech, powered by trust, and loved by customers. Every month, we engage 45 million app users and support over 73 million active listings. To help users seamlessly navigate this vast inventory, we've integrated recommendation systems across multiple touchpoints in all our platforms. These

recommendations are powered by the recommendations platform, which is responsible for delivering personalized suggestions across various contexts. Most, if not all, of these are served through a dedicated recommendations service.

Over the past few months, we've built and scaled this system within the data team, successfully shifting the ownership from a shared backend service to a service fully owned by the team to gain greater autonomy and flexibility. The team decided to build it with Python, as it is the go-to language for the data and machine learning team and is the most widely used language within both the team and the broader domain. While Python allows for rapid development and prototyping, working at scale has surfaced several challenges and trade-offs. It hasn't been an easy journey, but it's one that's taught us a lot and significantly matured our infrastructure and processes.

To Async or Not Async

The service consumes data from various sources: ScyllaDB, DynamoDB, and internal OLX services. A key optimization we implemented was converting these I/O operations into non-blocking calls by leveraging FastAPI's support for asynchronous programming.

FastAPI offers excellent documentation, including a dedicated guide to concurrency and async programming in Python. While we won't go into the weeds of Python's concurrency models, it's helpful to understand the key distinction between the two common approaches:

- **Threading** achieves concurrency by running multiple threads. When a thread performs a blocking I/O operation, Python can release the Global Interpreter Lock (GIL), allowing other threads to execute in the meantime.
- **Async I/O** is based on an event loop, where tasks voluntarily yield control while waiting for I/O operations to complete. This enables other tasks to run without blocking the entire application.

For I/O-bound workloads, async is generally the preferred approach; it's more lightweight, avoids thread management overhead, and thus scales better. However, async programming comes with a steeper learning curve and can introduce subtle

bugs or performance issues if not used carefully.

These trade-offs become especially relevant in FastAPI. The first place you'll encounter this trade-off is when defining path operations in FastAPI, where performance can hinge on how endpoints are declared: either with def (synchronous) or async def (asynchronous). This seemingly small decision has significant implications and the flow is encapsulated within this function and shown in Image1.

```
async def run_endpoint_function(
    *, dependant: Dependant, values: Dict[str, Any], is_coroutine: bool
) -> Any:
    assert dependant.call is not None, "dependant.call must be a function"

    if is_coroutine:
        return await dependant.call(**values)
    else:
        return await run_in_threadpool(dependant.call, **values)
```

Image1. FastAPI decides whether to run the path operation in the event loop or a thread pool based on function definition.

Declaring a route with def runs the function in a separate thread, which allows you to perform blocking I/O safely, without stalling the main event loop. However, this comes with the overhead of managing threads. Starlette, the ASGI framework FastAPI is built upon, uses a default thread pool of 40 tokens. On the other hand, declaring a route with async def runs it on the event loop, and all I/O operations must be awaited. This offers better performance if all the libraries, or wrappers, used are async-compatible.

It's also worth noting that this distinction between def and async def applies not only to route handlers but also to FastAPI's dependency injection system. Whether a dependency is defined synchronously or asynchronously affects how it's executed: either resolved in the thread pool or the event loop. Following the same rule, the

response validation happens either in a thread or the event loop. Therefore, understanding how many tasks are offloaded to the thread pool is important, as it's relatively easy to reach its limit and may require manual configuration. In reality, achieving a fully async service is tricky, especially when dealing with multiple data sources, legacy libraries, and a mix of sync and async libraries.

Our primary data source is ScyllaDB, but the official Python driver does not support the asyncio protocol (i.e, await and async def keywords). While it's technically possible to wrap the driver's future-style interface to make it awaitable, we chose to offload the query to a thread instead. The reason is simple: the driver internally already uses threads, so wrapping it in async didn't offer meaningful performance gains and could potentially block the event loop.

Even after making the ScyllaDB queries non-blocking, we saw **no improvement in service performance**. The reason is that operations against DynamoDB were still blocking the event loop. In Python, the standard way to interact with DynamoDB is through boto3, which, despite numerous community requests, is still not async-compatible. This forced us to choose between two options: either offload the blocking calls to a thread pool or adopt aioboto3, an async wrapper around boto3. After running several experiments in the wild, we found that aioboto3 offered better performance in terms of latency, the reason being that aioboto3 enables non-blocking I/O operations by leveraging Python's asyncio event loop with minimal overhead.

The other data source was other APIs. Fortunately, Python has strong support for asynchronous HTTP requests. While there's not much to add here, it's worth pointing to this GitHub issue in the httpx repository, which compares the performance and design trade-offs between httpx and aiohttp.

These optimizations resulted in a **latency reduction of over 40%** and a **more than 50% increase in throughput.**

Beyond Concurrency

By this point, all I/O operations had been converted to non-blocking calls. However, it became clear that the event loop was still being blocked at certain points. To

monitor this, we used New Relic's event loop diagnostics (Image2) and incorporated additional logging to pinpoint the bottlenecks.

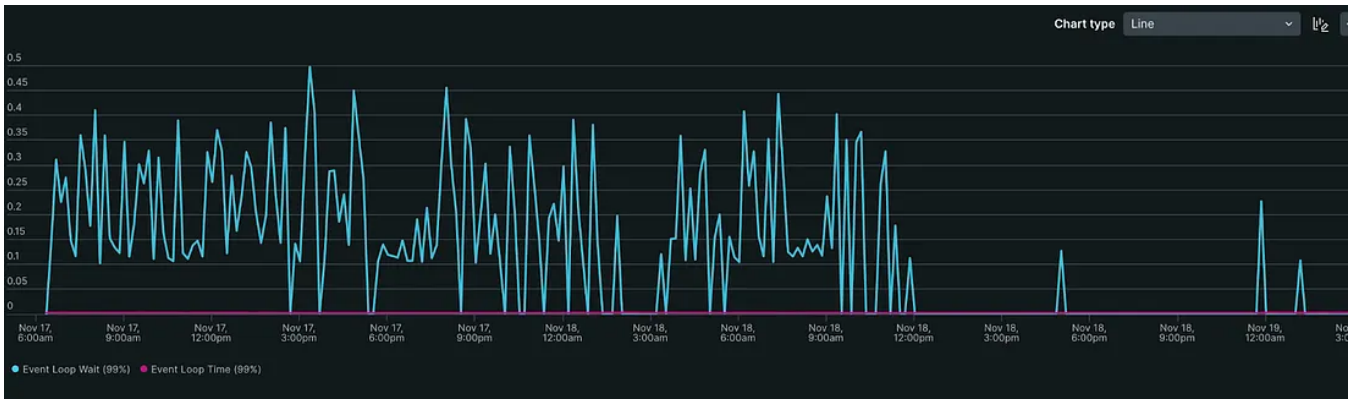

Image2. Time series from New Relic's event loop diagnostics. The blue line represents the time the event loop was waiting in seconds to regain control, illustrating the event loop before and after it becomes unblocked.

Optimizing Read Patterns

One of the first optimizations we made was avoiding fanning out queries to ScyllaDB. In scenarios where we wanted to provide personalized recommendations, the typical flow involves querying the data source to fetch the most recent user interactions, and then for each interaction query ScyllaDB to retrieve the most similar items. This created a fan-out pattern (Image3), one query per interaction, where even though the queries were concurrent, they were not run in parallel.

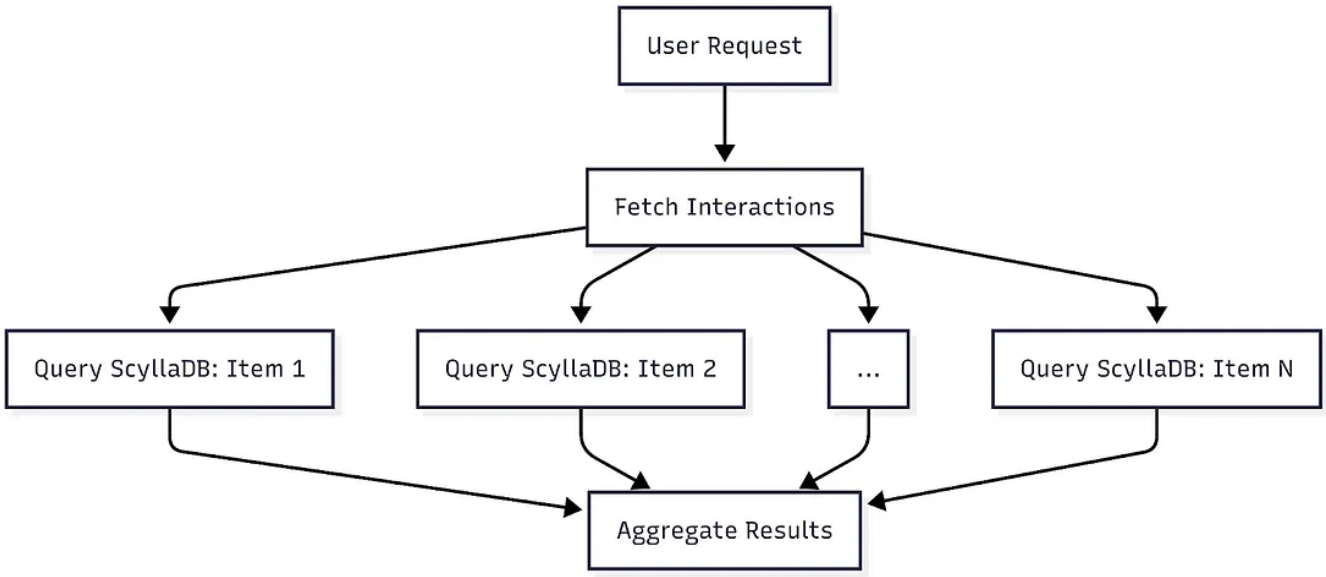

Image3. Diagram showing fan-out query pattern: a single request leads to multiple ScyllaDB queries based on user interactions.

To address this, we experimented with batching the queries into a single request to ScyllaDB. This significantly reduced overhead by minimizing network round-trips and easing the load on both the driver and the event loop. Most importantly, it helped prevent event loop stalls. As shown in Image2, this change led to a clear reduction in event loop blocking, ultimately **reducing p99 latency by over 50%**.

Minimizing Data Validation: Saying Goodbye to Pydantic

Data validation in FastAPI when using Pydantic takes place in three distinct stages: request input parameters, request output, and object creation. While this approach ensures robust data integrity, it can introduce inefficiencies. For instance, if an object is created and then returned, it will be validated multiple times: once during instantiation and again during response serialization. The situation can worsen if the object is re-created or used to construct another similar object, leading to redundant validations that consume unnecessary resources.

Get Jordi Esteve Sorribas's stories in your inbox

Join Medium for free to get updates from this writer.

Subscribe

Remember me for faster sign in

This redundancy might be acceptable when the objects being validated are small and simple, or when the Pydantic models themselves are lightweight. However, in the recommendation service, the system often needs to handle large volumes of data, several thousand ad ids plus metadata. In these cases, validating the same object more than once or a similar form becomes inefficient and can hurt performance.

In the plot above it shows that for a significant fraction of requests, the duration of building the Pydantic object took a significant amount of time. This wasn't just an issue for the ongoing request, it also impacted all other concurrent requests. Given

our use case, where we control most of the data we consume, we decided to drop Pydantic everywhere except for the input request parameters. We switched to using dataclasses with slots, which **drastically reduced latency by more than 30%.**

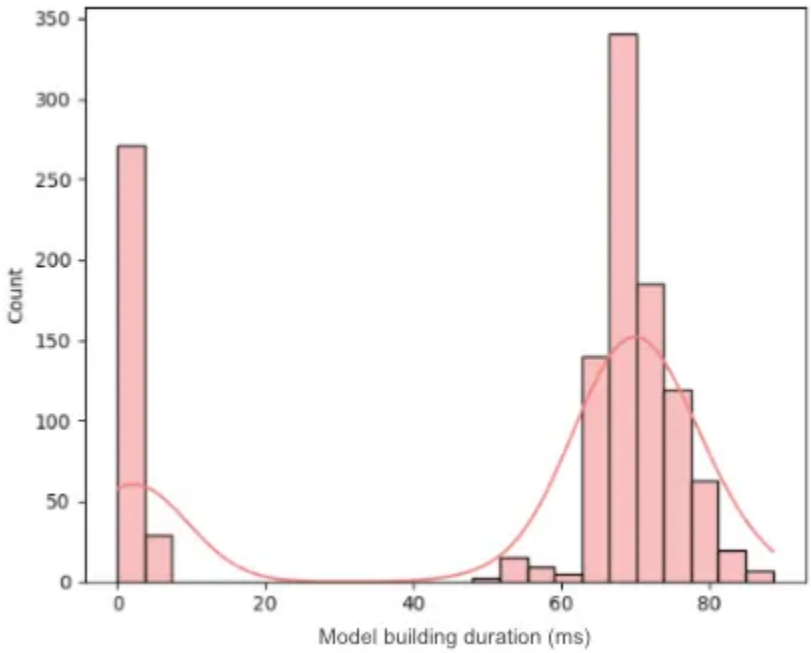

Image4. Distribution of time spent building the Pydantic model.

As a final note, it's important to highlight that performance can be affected not just by the size of the data being processed, but also by overly complex Pydantic models and type annotations. For example, consider a response model defined as a Union[A, B]. In this case, FastAPI (via Pydantic) will validate first against model A, and if it fails against model B. If A and B are deeply nested or complex, this leads to redundant and expensive validation, which can negatively impact performance.

Latency Roulette: When Garbage Collector (GC) Enters the Game

Even after removing Pydantic, we continued to see unexpected latency spikes during model construction. The bimodal distribution shown in Image5 was particularly puzzling. The input content, object size, and schema remained constant, yet performance varied significantly. After ruling out obvious causes, we realized that something outside our direct control might be at play: the **garbage collector (GC)**. Disabling it entirely wasn't an option, but tuning its behavior was.

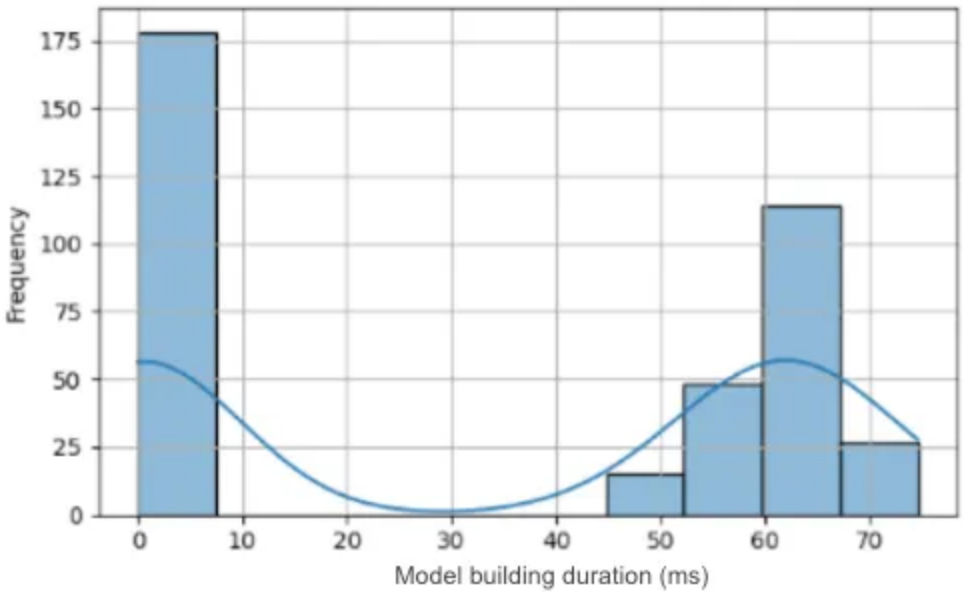

Image5. Distribution of time spent building the model using dataclasses. While the improvements over Pydantic (Image4) are notable, there's still room for further optimization.

The GC is triggered when the number of allocations minus the number of deallocations exceeds certain thresholds for each generation. Python's GC organizes objects into three generations (0, 1, and 2). New objects start in generation 0. If they survive a collection, they get promoted to the next generation, more details here. Given the bimodal distribution in the above image, we hypothesized that the default Python GC settings might be suboptimal for our high-throughput needs.

By increasing the generation 0 threshold and adjusting the thresholds for generations 1 and 2, we aimed to reduce the frequency of garbage collection, thereby lowering tail latencies without significantly increasing memory consumption. However, this requires careful tuning: too infrequent collections can cause p99 latencies to spike, as a larger number of objects accumulate before collection.

To test this hypothesis, we ran a series of local load tests comparing the default GC settings with optimized configurations. Our goal was to strike a balance between the number of GC runs and the duration of each run. By increasing the generation 0 threshold from 700 to 7000 objects and raising the thresholds for generations 1 and 2 from 10 to 20, we observed improved tail latencies and a reduction in the frequency

of micro-pauses.

After deploying this configuration in production, we saw a **20% overall latency reduction in our service**. More notably, the latency for homepage recommendation requests, which return the most data, improved dramatically, with **p99 latency dropping from 52ms to 12ms** (Image6). Shout out to **Denis Belyakov** for running the load tests and finding the best GC parameters!

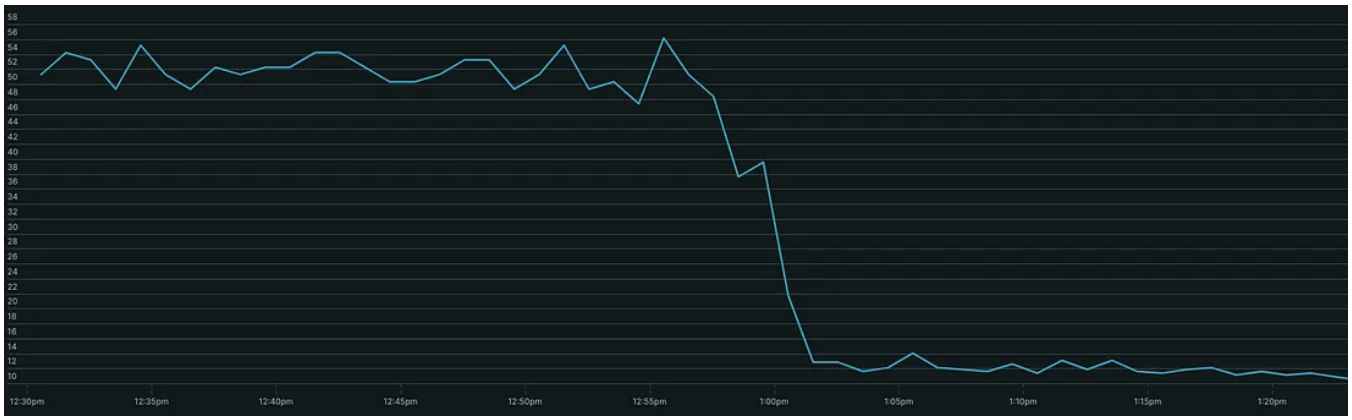

Image6. Latency reduction of homepage recommendations. The y-axis is latency in milliseconds and x-axis is the time.

Deployment and Infrastructure

One of the standout features of ScyllaDB is its shard-per-core awareness, which allows clients to route queries directly to the relevant CPU core on the target node. To validate this behavior, we added logging to check connections across all shards and noticed that one shard was never being connected to, indicating an issue with shard connectivity (Image7).

```
def _print_cluster_stats(cluster):
    if cluster.is_shard_aware():
        logger.info("Connected to a shard aware cluster")
        logger.info("Shard aware stats: %s", cluster.shard_aware_stats())
    else:
        logger.warning("Not connected to a shard aware cluster")
```

Image7. Snipped code used to monitor shard aware connections to the Scylla cluster

Independent of the recommendation service, we enhanced the deployment strategy by using topology spread constraints and node selectors.

Learnings and Conclusions

In this journey, we learnt a lot about FastAPI and the Python ecosystem, Scylla and other technical dimensions. While we gained valuable technical insights and reached our goals, a few key takeaways stood out:

- **Debugging and reasoning in a concurrent world under the reign of the GIL is not easy.** You might have optimized 99% of your request, but a rare operation, happening just 1% of the time, can still become a bottleneck that drags down overall performance.
- **FastAPI and Python enable rapid development and prototyping**, but at scale, it's crucial to understand what's happening under the hood. You'll likely need to run multiple rounds of profiling and optimization. At that point, it's worth questioning whether switching to a different language might be more efficient.
- **Load test like there's no tomorrow.** You'll uncover unexpected performance issues only when you simulate real-world traffic and usage patterns. Internally, we used vegeta to run these load tests with great support from our SREs.
- **The majority of these steps were executed in the presented order**. However,

there may be confounding variables that were overlooked, for example, retaining Pydantic and different GC thresholds might have produced similar results.

- **Start small, test, and extend.** I can't stress enough how important it is to start with a PoC, evaluate it, address the problems, and move forward. Down the line, it is very difficult to debug a fully featured service that has scalability problems.

Python Machine Learning Scalability Concurrency Optimization

Follow

Published in OLX Engineering

2.8K followers · Last published Mar 18, 2026

OLX is a global leader in facilitating trade. It builds leading marketplace ecosystems enabled by tech, powered by trust, and loved by its customers.

Follow

Written by Jordi Esteve Sorribas

23 followers · 3 following

MSc Data Science @ University of Edinburgh

No responses yet

Write a response

What are your thoughts?